11.1-Logistic Regression with Hyperparameter Tuning

August 3, 2025

1 Implementation Of Logistic Regression

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
```

[2]:
```
## creating the dataset using sklearn library
from sklearn.datasets import make_classification
```

[3]:
```
## Creating the dataset

X, y = make_classification(n_samples=1000, n_features=10, n_classes=2,␣
 ↪random_state=42)
'''
So here:
n_samples is my total datapoint = 1000
n_features are the independent features or input = 10
n_classes = 2 (for binary)

'''
```

[3]:
```
'\nSo here:\nn_samples is my total datapoint = 1000\nn_features are the
independent features or input = 10\nn_classes = 2 (for binary)\n\n'
```

[4]:
```
X.shape, y.shape
```

[4]:
```
((1000, 10), (1000,))
```

[5]:
```
df = pd.DataFrame(X)
```

[6]:
```
df.head()
```

[6]:
```
          0         1         2         3         4         5         6  \
0  0.964799 -0.066449  0.986768 -0.358079  0.997266  1.181890 -1.615679
```

```
1 -0.916511 -0.566395 -1.008614  0.831617 -1.176962  1.820544  1.752375
2 -0.109484 -0.432774 -0.457649  0.793818 -0.268646 -1.836360  1.239086
3  1.750412  2.023606  1.688159  0.006800 -1.607661  0.184741 -2.619427
4 -0.224726 -0.711303 -0.220778  0.117124  1.536061  0.597538  0.348645

          7         8         9
0 -1.210161 -0.628077  1.227274
1 -0.984534  0.363896  0.209470
2 -0.246383 -1.058145 -0.297376
3 -0.357445 -1.473127 -0.190039
4 -0.939156  0.175915  0.236224
```

[7]:
```
df1 = pd.DataFrame(y)
```

[8]:
```
df1[0].value_counts()
```

[8]:
```
0
0    501
1    499
Name: count, dtype: int64
```

[9]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,␣
 ↪random_state=42)
```

[10]:
```
## Model training

from sklearn.linear_model import LogisticRegression

logistic = LogisticRegression()
```

[11]:
```
logistic.fit(X, y)
```

[11]:
```
LogisticRegression()
```

[12]:
```
y_pred = logistic.predict(X_test)
y_pred
```

[12]:
```
array([0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0,
       1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0,
       0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1,
       0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0,
       0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
       1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,
       1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1,
       0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1,
       1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1,
```

```
       1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
       0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1,
       0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1,
       1, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0])
```

[13]:
```
# logistic.predict_proba(X_test)
```

[14]:
```
## Performance matrix

from sklearn.metrics import accuracy_score, confusion_matrix,␣
 ↪classification_report
```

[15]:
```
score = accuracy_score(y_test, y_pred)
score
```

[15]:
```
0.8466666666666667
```

[16]:
```
cm = confusion_matrix(y_test, y_pred)
cm
```

[16]:
```
array([[116,  19],
       [ 27, 138]], dtype=int64)
```

[17]:
```
cr = classification_report(y_test, y_pred)
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.81      0.86      0.83       135
           1       0.88      0.84      0.86       165

    accuracy                           0.85       300
   macro avg       0.85      0.85      0.85       300
weighted avg       0.85      0.85      0.85       300
```

2 Hyperparameter Tuning And Cross Validation

[18]:
```
model = LogisticRegression()

# penalty = ['l1', 'l2', 'elasticnet']  # here l1 is lasso regression, l2 is␣
 ↪ridge regression and other is elasticnet regression
# c_values = [100, 10, 1, 0.1, 0.01]
# solver = ['newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga']
```

[19]:
```
## Creating a dictionary for parameter tuning
# params = dict(penalty=penalty, C = c_values, solver=solver
```

```
params = [
    # Solvers that only support l2
    {'solver': ['newton-cg', 'lbfgs', 'sag'], 'penalty': ['l2'], 'C': [100, 10,␣
 ↪1, 0.1, 0.01]},

    # liblinear supports l1 and l2
    {'solver': ['liblinear'], 'penalty': ['l1', 'l2'], 'C': [100, 10, 1, 0.1, 0.
 ↪01]},

    # saga supports all penalties
    {'solver': ['saga'], 'penalty': ['l1', 'l2', 'elasticnet'], 'C': [100, 10,␣
 ↪1, 0.1, 0.01]}
]
```

3 GridSearchCV

[20]:
```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
cv = StratifiedKFold(n_splits=5)
grid = GridSearchCV(estimator=model, param_grid=params, scoring='accuracy',␣
 ↪cv=cv, n_jobs=-1)

# Fit the model
grid.fit(X_train, y_train)

# Get the best score
print(grid.best_score_)
```

```
0.8785714285714287
```

[21]:
```
grid.best_params_
```

[21]:
```
{'C': 0.01, 'penalty': 'l2', 'solver': 'newton-cg'}
```

[22]:
```
y_pred = grid.predict(X_test)
```

[23]:
```
score = accuracy_score(y_pred, y_test)
print(score)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_pred, y_test))
```

```
0.8533333333333334
              precision    recall  f1-score   support

           0       0.79      0.92      0.85       135
```

```
           1       0.92      0.80      0.86       165

    accuracy                           0.85       300
   macro avg       0.86      0.86      0.85       300
weighted avg       0.86      0.85      0.85       300

[[124  33]
 [ 11 132]]
```

[24]:
```
from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score

best_model = grid.best_estimator_

# Cross-validation accuracy on training data
cv_scores = cross_val_score(best_model, X_train, y_train, cv=5)
print("Mean CV Accuracy on Train:", np.mean(cv_scores))

# Accuracy on test data
test_accuracy = accuracy_score(y_test, best_model.predict(X_test))
print("Test Accuracy:", test_accuracy)
```

```
Mean CV Accuracy on Train: 0.8785714285714287
Test Accuracy: 0.8533333333333334
```

Why the Scores Are Different:

grid.best_score_ = 87%: This is the average accuracy across 5-fold cross-validation on the training set only. It's used to find the best hyperparameters.

classification_report = 85%: After GridSearchCV found the best model, you tested it on new, unseen test data — naturally, performance can drop slightly here.

4 RandomizedSearchCV

[25]:
```
from sklearn.model_selection import RandomizedSearchCV
```

[26]:
```
rand = RandomizedSearchCV(estimator=model, param_distributions=params,␣
 ↪scoring='accuracy', cv=5, n_jobs=-1)
```

[27]:
```
rand.fit(X_train, y_train)
```

[27]:
```
RandomizedSearchCV(cv=5, estimator=LogisticRegression(), n_jobs=-1,
                   param_distributions=[{'C': [100, 10, 1, 0.1, 0.01],
                                         'penalty': ['l2'],
                                         'solver': ['newton-cg', 'lbfgs',
                                                    'sag']},
                                        {'C': [100, 10, 1, 0.1, 0.01],
                                         'penalty': ['l1', 'l2'],
```

```
                                          'solver': ['liblinear']},
                                         {'C': [100, 10, 1, 0.1, 0.01],
                                          'penalty': ['l1', 'l2', 'elasticnet'],
                                          'solver': ['saga']}],
                   scoring='accuracy')
```

[28]:
```
rand.best_estimator_
```

[28]:
```
LogisticRegression(C=0.1, penalty='l1', solver='liblinear')
```

[29]:
```
rand.best_score_
```

[29]:
```
0.8742857142857143
```

[30]:
```
y_pred = rand.predict(X_test)
score = accuracy_score(y_pred, y_test)
print(score)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_pred, y_test))
```

```
0.8533333333333334
              precision    recall  f1-score   support

           0       0.79      0.91      0.85       135
           1       0.92      0.81      0.86       165

    accuracy                           0.85       300
   macro avg       0.86      0.86      0.85       300
weighted avg       0.86      0.85      0.85       300

[[123  32]
 [ 12 133]]
```

Same accuracy difference as GridSearchCV